

1. Iterate until $\Delta L(\vec{w}) < \text{threshold}$.

Measure on validation set.

2. Iterate until $\Delta L(\vec{w}) < \text{threshold}$.

for N iterations.

"patience"

Measure on validation set.

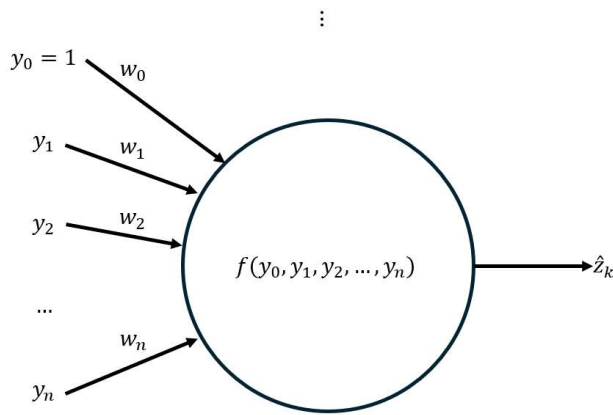
3. Stop after a fixed number of steps.

Backpropagation

Example $y = h(g(f(x)))$

$$\frac{\partial y}{\partial x} = h'(g(\underline{f(x)})) \cdot g'(\underline{f(x)}) \cdot f'(x)$$

Store derivatives and other intermediate quantities in memory and re-use them.



Consider k^{th} artificial neuron in output layer.
 Suppose this is part of a regression task
 with sum of squares error loss.

$$L = \frac{1}{2} \sum_d \sum_l (z_l - \hat{z}_l)^2$$

↑
summation over
training instances
↑
summation over
components

We want $\frac{\partial L}{\partial w_j}$ for all j .

$$s_k = \sum_l w_l y_l$$

$$\hat{z}_k = a(s_k)$$

$$\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial s_k} \frac{\partial s_k}{\partial w_j}$$

$\frac{\partial s_k}{\partial w_j} = y_j$

$$\frac{\partial L}{\partial s_k} = \frac{\partial L}{\partial \hat{z}_k} \frac{\partial \hat{z}_k}{\partial s_k}$$

$$\frac{\partial L}{\partial s_k} = - \sum_d (z_k - \hat{z}_k) a'(s_k)$$

$$\frac{\partial L}{\partial w_j} = \left[- \sum_d (z_k - \hat{z}_k) a'(s_k) \right] y_j$$

$\frac{\partial L}{\partial s_k}$ will be useful for computing partial derivatives in other layers of the network